Chapter 1

Introduction to Modeling

Extracted from the textbook:
Object Oriented Modeling and Design with UML

Spring 2023\2024

What is a Model?

- Software Modeling plays an important role in software engineering.
  - Current trends: MDSE (Model Driven SE).

- A model is an abstract, simplified or incomplete description/representation of a part of the real word

Modeling

- It is the process of creating models.
- Modeling is often practiced in all engineering disciplines.
- Modeling requires a power of **abstraction:**
  - Persons who cannot THINK ABSTRACTLY fail to learn modeling skills.

Abstraction

- Selective examination of certain aspects of a problem or system.
- Goal:
  - Isolate those aspects that are important for some purpose and suppress those aspects that are not important: **essential** versus **details.**
- Abstraction must always be for **some purpose**.
- Many different abstractions of the same system are possible **depending on their purposes.**

Abstraction

- Abstractions are incomplete and inaccurate.
- Don’t search for absolute truth but for adequacy for some purpose.
- **Agood modell capturesa problem’s crucial aspects andignorese the rest.**

Why Modeling?

1. Communication with customers

- Product engineers/architects build models to show to customers.
- Prototypes/Mock-ups: demonstration that imitates some of or all of the external behavior of a system

Why Modeling?

2. Reduction of complexity:

- The main reason for modeling is to deal with systems that are too complex to understand directly.
- The human mind copes with only a small amount of information at one time.
- Models reduce complexity by separating out afewf important things to deal with at a time.

What to model?

- **Real World Modeling:**
  - **Usage1: facilitate understanding of a problem in the real world without the intention of developing an application based on the model.**
    - **Example1: Business process reengineering (business improvement goal)**
    - **Example2: Requirement elicitation step (software development)**
  - **Usage 2: Develop an application that simulates the real world.**
    - **Example: Aircraft flight simulation**

What to model?

- **“Cyber” Real World."العالم الحقيقي "السيبراني**
  - **Usage: Develop an application that automates business in the real world**
  - **Examples:**
    - **Automating University library.**
    - **Automating Student Registration نظام التسجيل عبارة عن. موديل لعملية التسجيل**

Modeling in SE

- **MDSE** is based on the idea that SE life cycle phases produce and consume models.
- Models produced during a **given phase** will be consumed and used in the **next phases.**
- **Artifacts:** requirement models
- analysis models,
- design models,
- testing models,
- implementation model (code).

Uses of Models in SE

- **Models as sketches**:رسومات
- Developers find it useful to sketch **descriptions of requirements, design,** or deployment concepts on **whiteboards or paper** when discussing their ideas with other developers or customer representatives.
- **Models as analysis artifacts**: **Developers build analyzable models to check specified properties** (e.g., consistency and satisfiability properties), to predict implementation qualities (e.g., performance), or to simulate implemented behavior
- **Models as the basis for code generation or a mixture of software artifacts**: Models can be built to generate implementations, test cases, deployment or software configuration scripts, or other software artifacts.

Modeling in SE

- It is useful to model a system from different but related viewpoints.
- Each viewpoint captures important aspects of the system.
- A combination of these viewpoints must capture a complete description of the system.
- يجب أن تحتوي مجموعة وجهات النظر هذه على وصف كامل للنظام.
- Each model is intended to describe one aspect/viewpoint of a system but contains references to the other models.
- يهدف كل نموذج إلى وجهة نظر واحدة للنظام ولكنه يحتوي على إشارات إلى النماذج الأخرى

Which viewpoints to model?

- **Functional**/process aspect.
- **Structural**/Data aspect
- **Behavioral**/Dynamic aspect
- **Interaction**/collaboration aspect

How to express Models?

- Need of **modeling languages** to express models.
- Modeling languages should be simple enough to be comprehensible and usable by modelers.
- Modeling languages should be **communicative** and **abstract.**

Typology of modeling languages

- Various criteria may be used to make a classification of modeling languages:
  - **Textual** versus **diagram-based syntax.** **نصوص او رسومات**
  - **General purpose** or **Domain modeling languages** **مخصص او عام.**

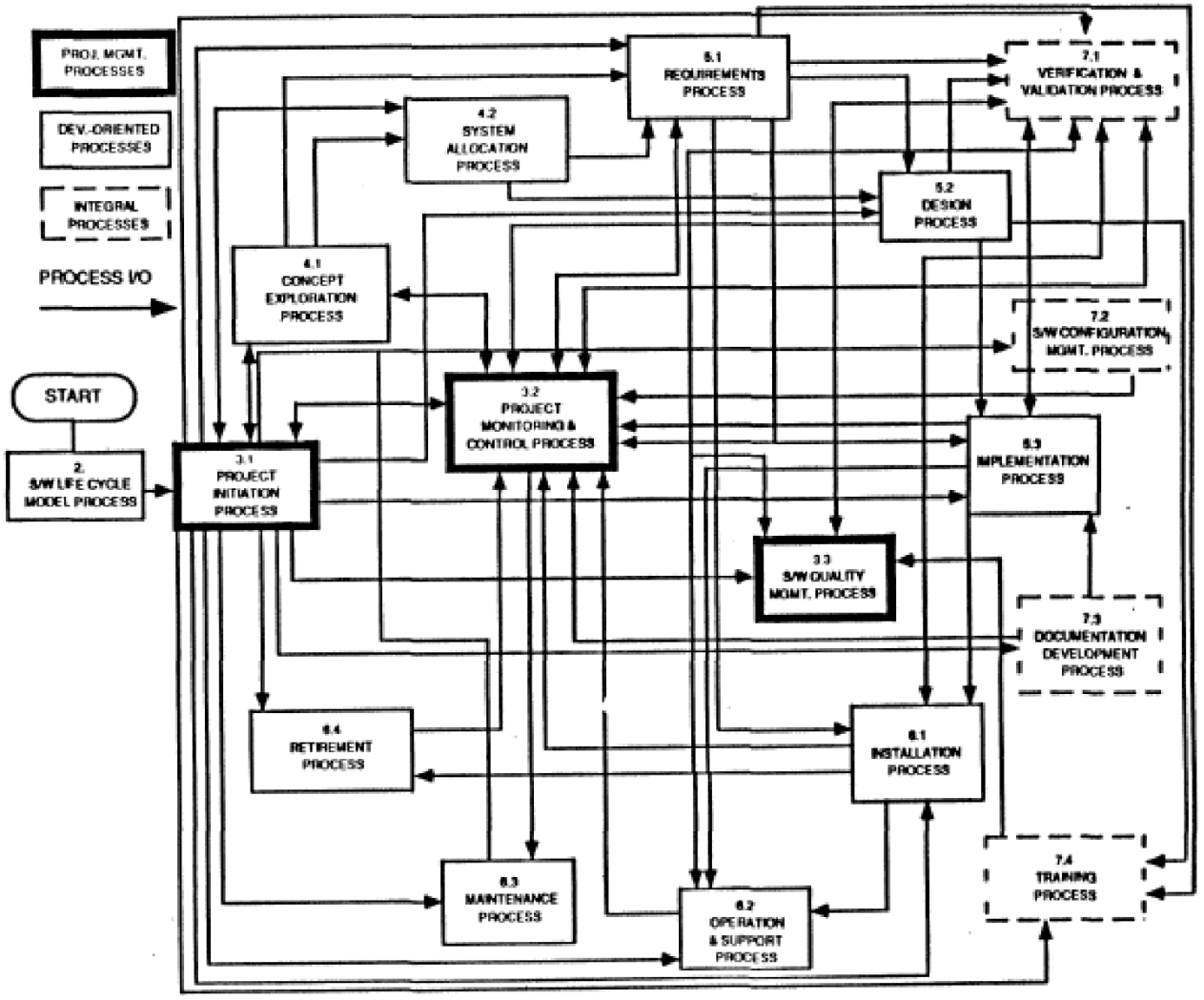
Part 2

Object-Oriented Software Engineering
Using UML, Patterns, and Java

Chapter 1, Modeling



What is the problem with this

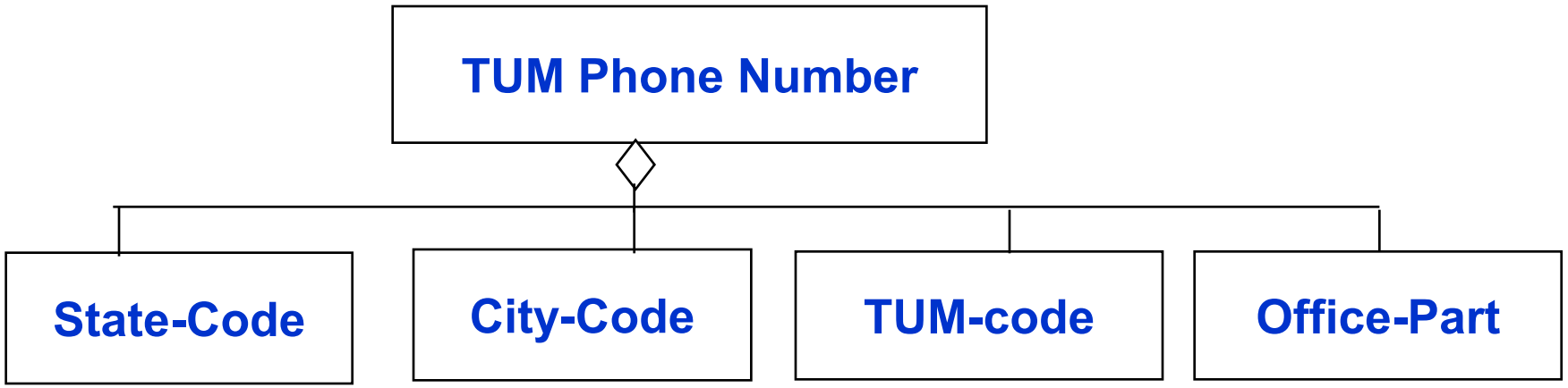


Abstraction

- Complex systems are hard to understand
  - The 7 +- 2 phenomena
    - Our short term memory cannot store more than 7+-2 pieces at the same time -> limitation of the brain
    - Phone Number: 498928918204
- Chunking:
    - Group collection of objects to reduce complexity
    - 4 chunks:
      - State-code, city-code, TUM-code, Office-Part

Abstraction

- Complex systems are hard to understand
  - The 7 +- 2 phenomena
    - Our short term memory cannot store more than 7+-2 pieces at the same time -> limitation of the brain
    - TUM Phone Number: 498928918204
- **Chunking**
    - Group collection of objects to reduce complexity
    - State-code, city-code, TUM-code, Office-Part



Abstraction

- Abstraction allows us to ignore unessential details
- Two definitions for abstraction:
  - Abstraction is a *thought process* where ideas are distanced from objects
    - Abstraction as activity
  - Abstraction is the *resulting idea* of a thought process where an idea has been distanced from an object
    - Abstraction as entity
- Ideas can be expressed by models



We use Models to describe Software Systems

- Object model: What is the structure of the system?
- Functional model: What are the functions of the system?
- Dynamic model: How does the system react to external events?
- **System Model: Object model + Functional model + Dynamic model**

Model

- A model is an abstraction of a system
  - A system that no longer exists
  - An existing system
    - A future system to be built.





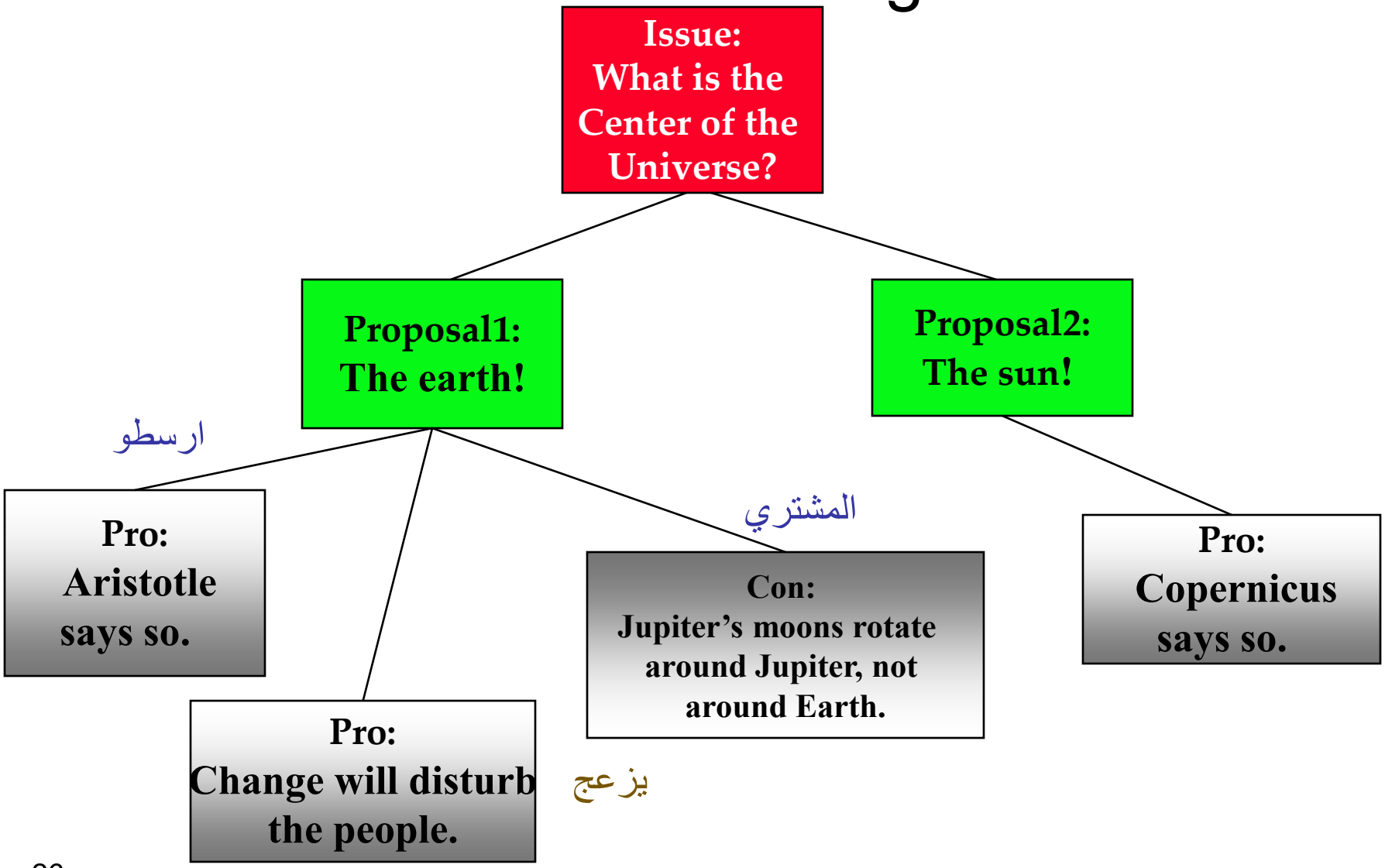
Other models used to describe Software System Development

- **Task Model:**
  - **PERT Chart:** What are the dependencies between tasks?
  - **Schedule:** How can this be done within the time limit?
  - **Organization Chart:** What are the roles in the project?

Issues Model:

- What are the open and closed issues?
- What blocks me from continuing?
- What constraints were imposed by the client?
- What resolutions were made?
- These lead to action items

Issue-Modeling



Technique to deal with Complexity
Decomposition

- A technique used to master complexity (“divide and conquer”)
- Two major types of decomposition
  - Functional decomposition
  - Object-oriented decomposition

Functional decomposition

- The system is decomposed into modules
- Each module is a major function in the application domain
- Modules can be decomposed into smaller modules.

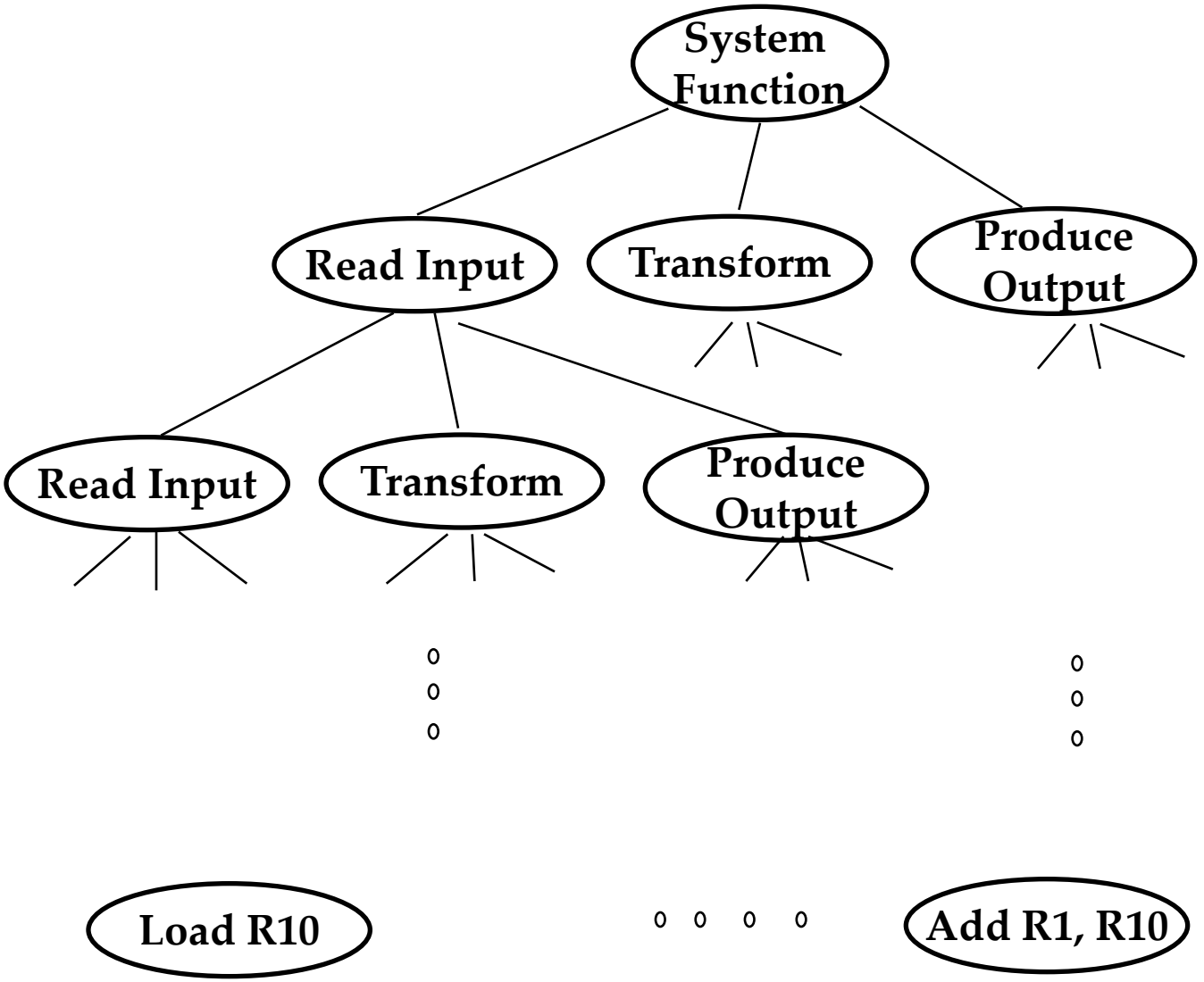
Decomposition (cont'd)

- **Object-oriented decomposition**
  - The system is decomposed into classes ("objects")
  - Each class is a major entity in the application domain
  - Classes can be decomposed into smaller classes.

Which decomposition is the right one?

- Object-oriented vs. functional decomposition

Functional Decomposition



Top Level functions

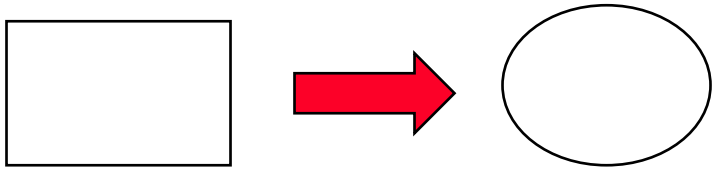
Level 1 functions

Level 2 functions

Machine instructions

Functional Decomposition

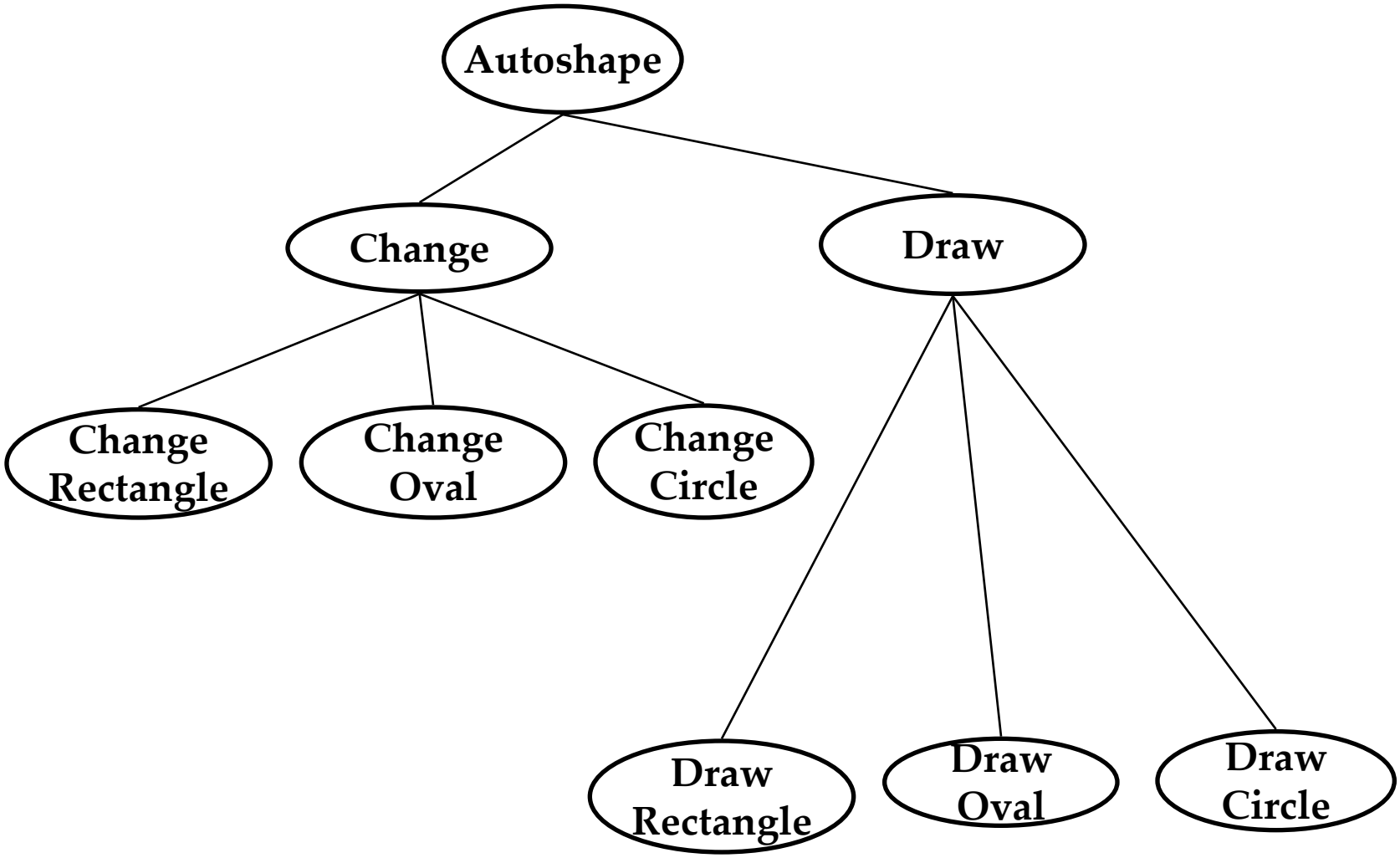
- The functionality is spread all over the system
- **Maintainer** must understand the whole system to make a single change to the system
- **Consequence:** للاهمية
  - The source code is hard to understand
  - **Source code is complex and impossible to maintain**
  - **User interface is often awkward and non-intuitive**



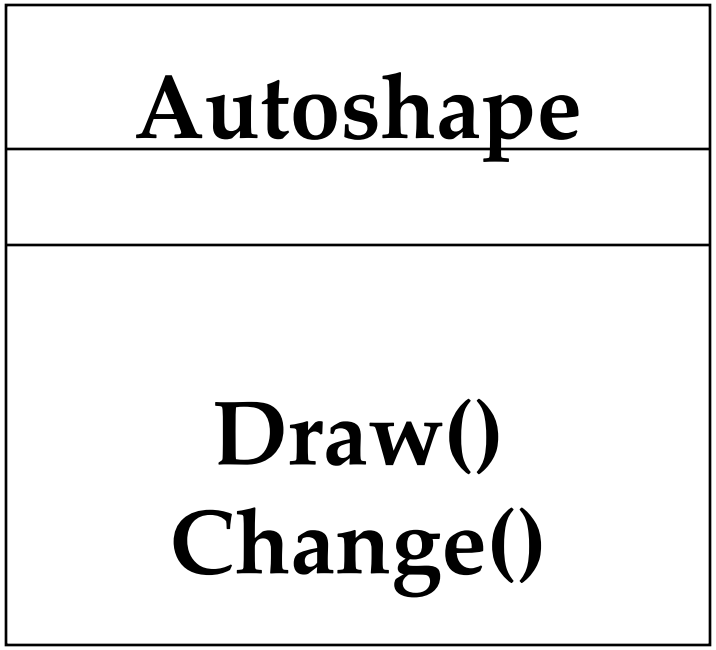
?

- **Example: Microsoft Powerpoint's Autoshapes**
  - **How do I change a square into a circle?**

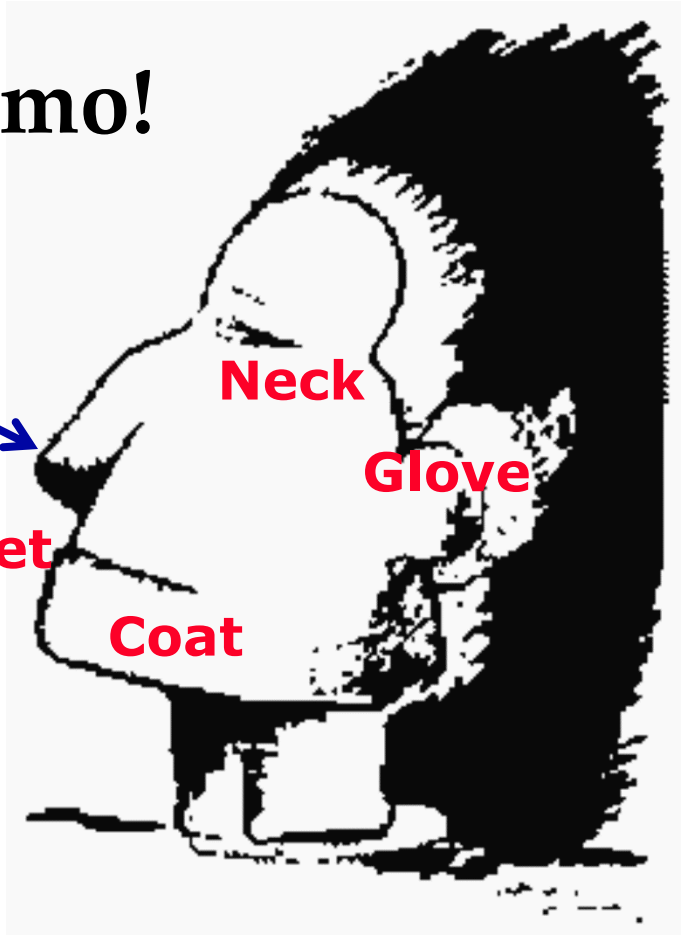
Functional Decomposition: Autoshape



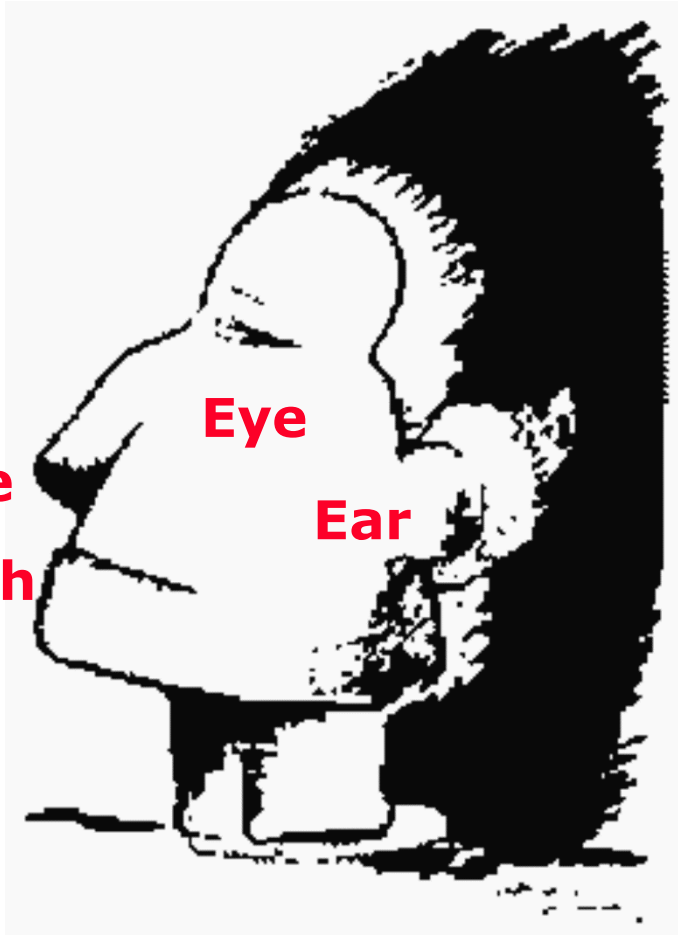
Object-Oriented View



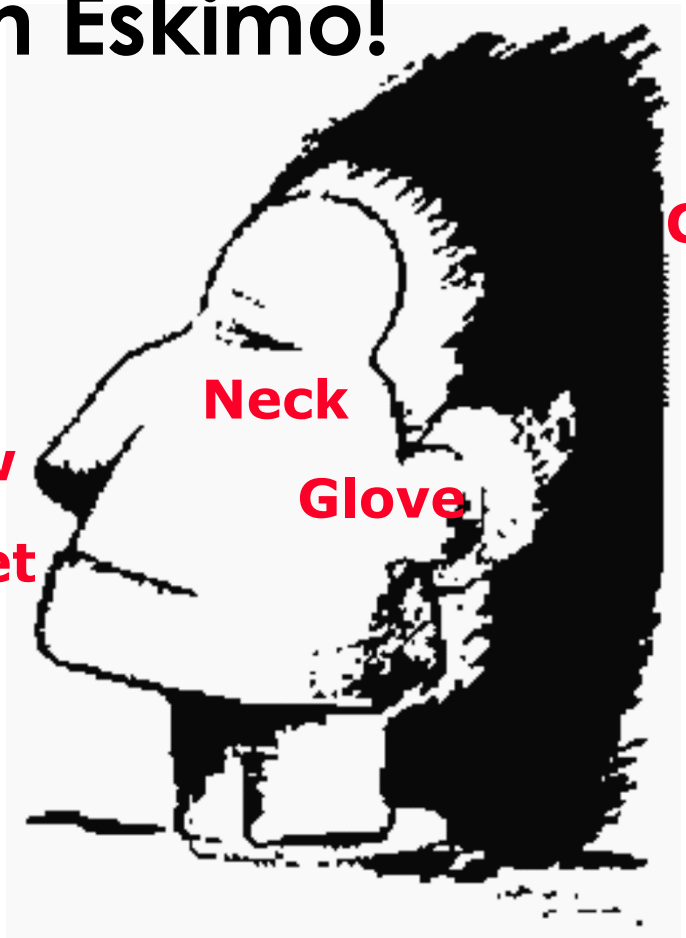
What is This?



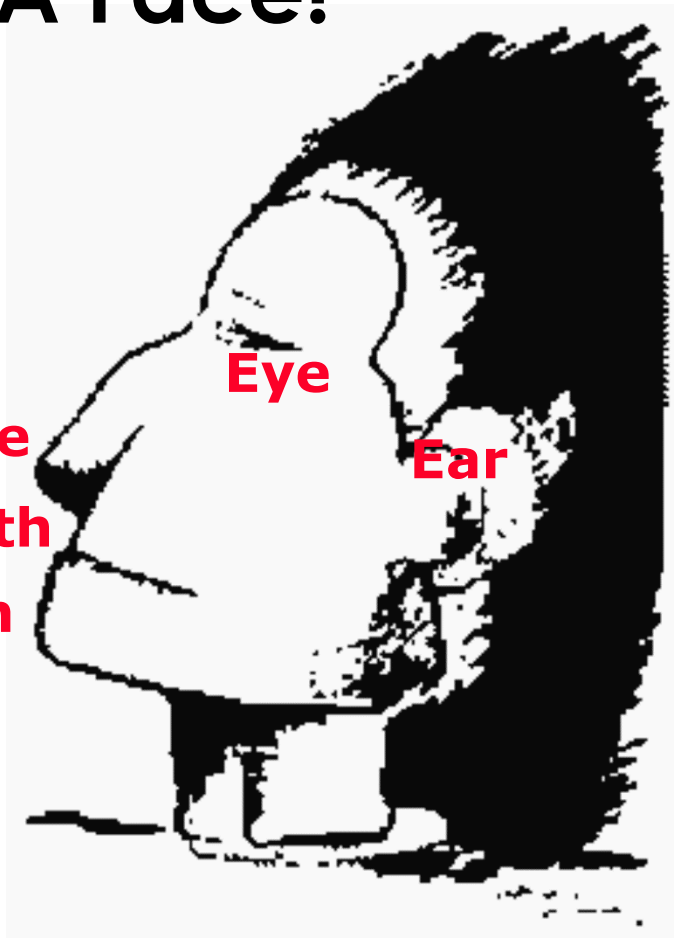
A Face!



An Eskimo!



A Face!



Class Identification

- **Basic assumptions:**
  - We can find the classes for a new software system: Greenfield Engineering
  - We can identify the classes in an existing system: Reengineering
  - We can create a class-based interface to an existing system: Interface Engineering



Class Identification (cont'd)

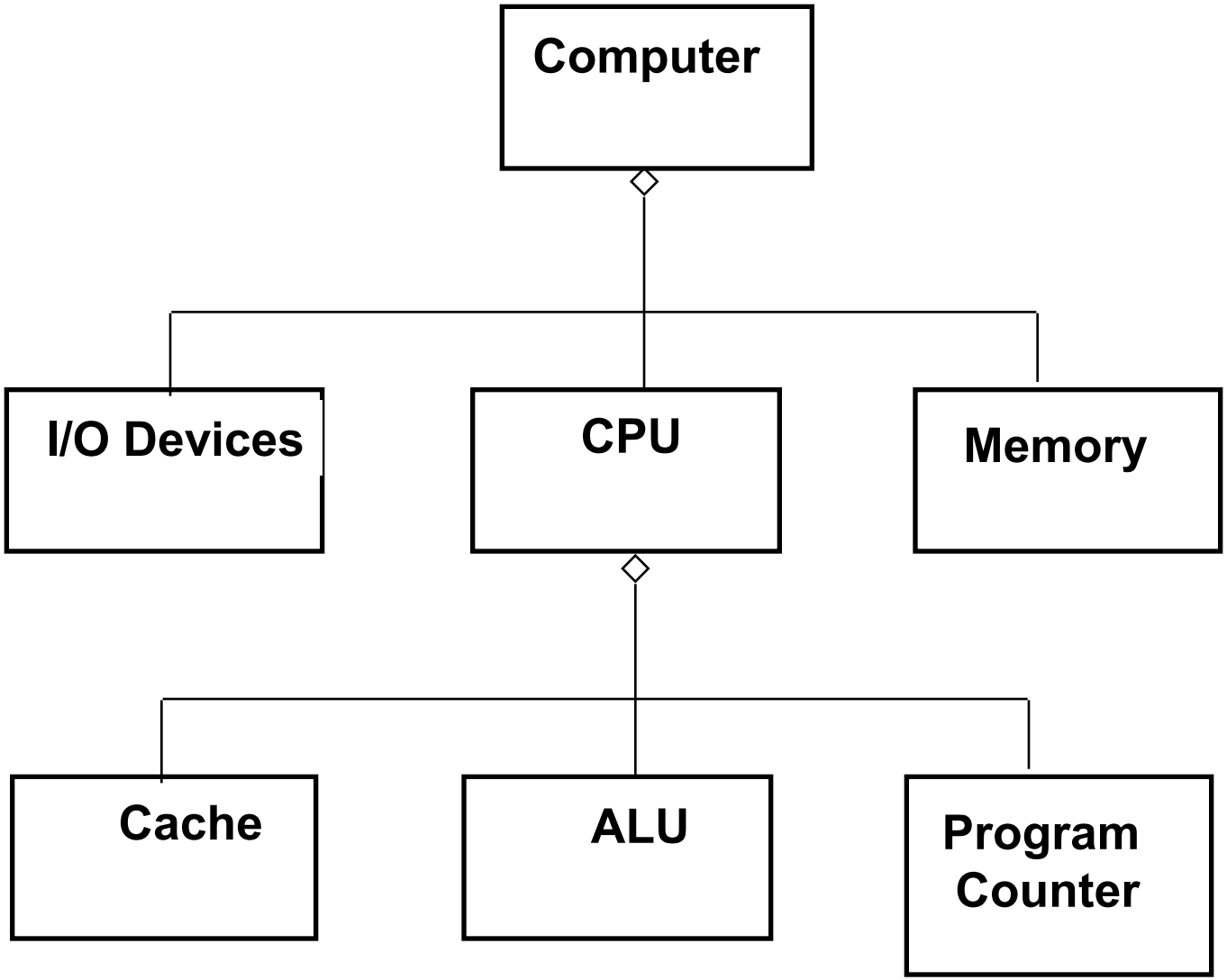
- **Why can we do this?**
  - Philosophy, science, experimental evidence
- **What are the limitations?**
  - Depending on the purpose of the system, different objects might be found
- **Critical**

  Identify the purpose of a system

Hierarchy

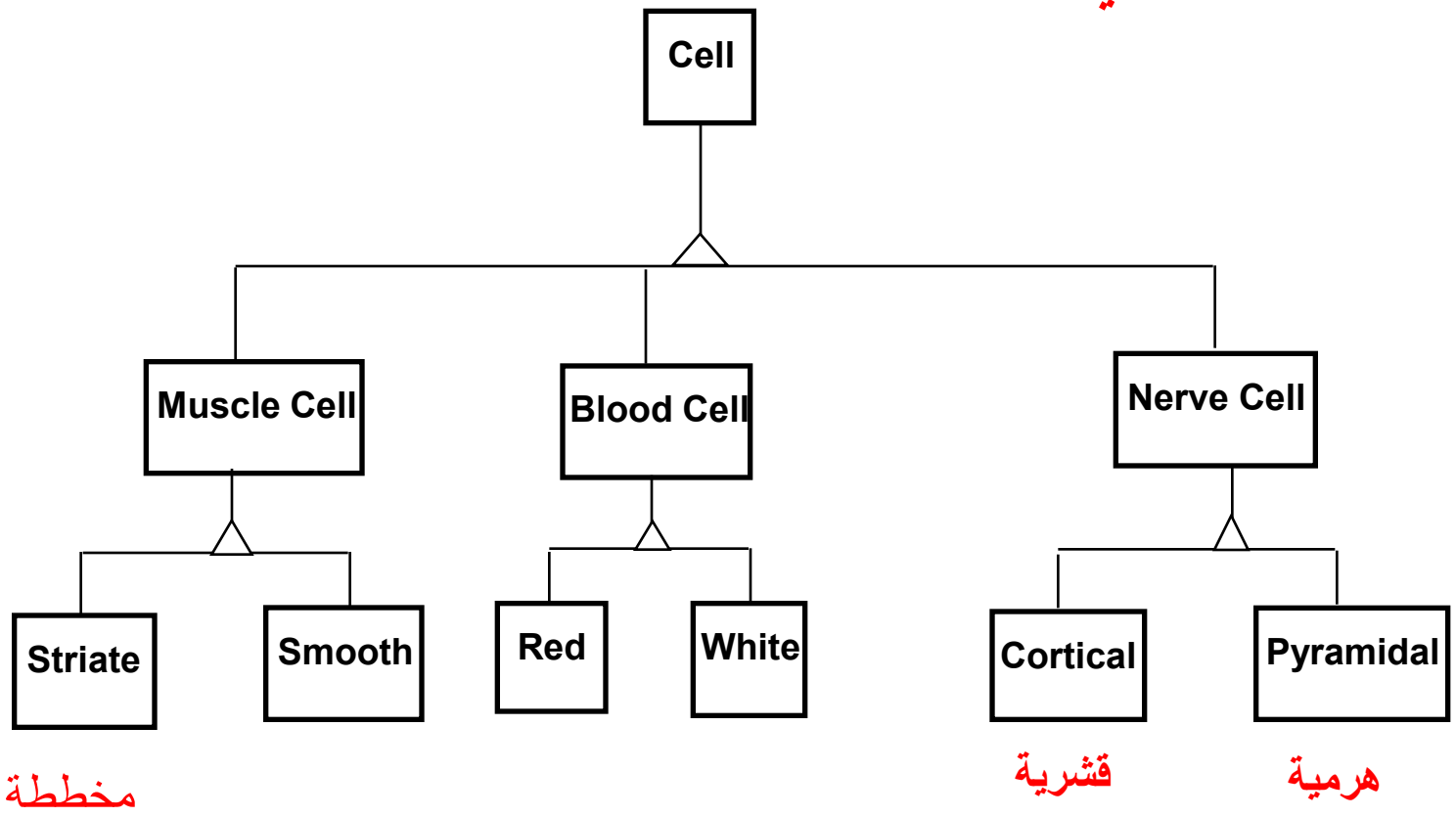
- So far we got abstractions
  - This leads us to classes and objects
  - “Chunks”
- Another way to deal with complexity is to provide relationships between these chunks
- One of the most important relationships is a hierarchy
- 2 special hierarchies
  - "Part-of" hierarchy
  - "Is-kind-of" hierarchy

Part-of Hierarchy (Aggregation) مكون اساسي



Is-Kind-of Hierarchy (Taxonomy)

التصنيف



Where are we now?

- Three ways to deal with complexity:
  - **Abstraction, Decomposition, Hierarchy**
- Object-oriented decomposition is a good
  - Unfortunately, depending on the purpose of the system, different objects can be found
- How can we do it right?
  - Start with a description of the functionality of a system
  - Then proceed to a description of its structure
- Ordering of development activities
  - **Software lifecycle**

- Karl Popper ("Objective Knowledge):
  - There is no absolute truth when trying to understand reality.
  - One can only build theories, that are "true" until somebody finds a counter-example
- **Falsification:** The act of disproving a theory or hypothesis
- The truth of a theory is never certain. We must use phrases like:
  - "by our best judgment", "using state-of-the-art knowledge"..استخدام أحدث المعارف

In software engineering any model is a theory:

We build models and try to find counter-examples by:

- Requirements validation,
- user interface testing,
- review of the design,
- source code testing,
- system testing, etc.

Testing: The act of disproving a model.

Concepts and Phenomena

- **Phenomenon**
  - An object in the world of a domain as you notice it
    - Examples: This lecture at 9:35,
    - **Concept**
  - Describes the common properties of phenomena
    - Example: All lectures on software engineering

A Concept is a 3-tuple:

- **Name:** The name differentiates the concept from other concepts.
- **Purpose:** Properties that determine if a phenomenon is a member of a concept.
- **Members:** The set of phenomena which are part of the concept.

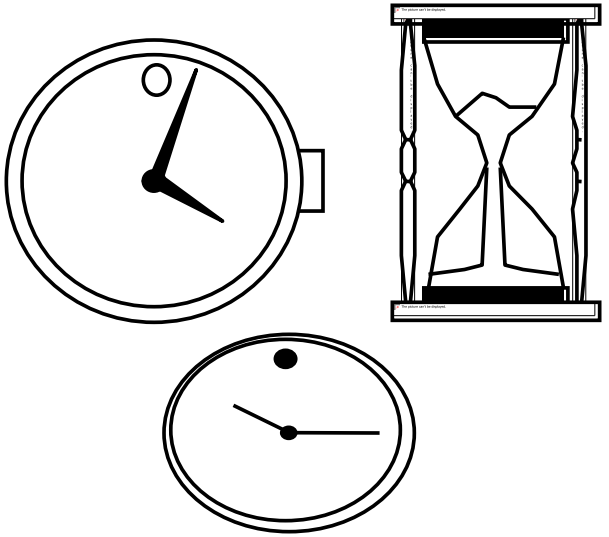
- **Abstract data type**
  - A type whose implementation is hidden from the rest of the system
- **Class:**
  - An abstraction in the context of object-oriented languages
  - A class encapsulates state and behavior
    - Example: Watch

Concepts, Phenomena, Abstraction and Modeling

Name **Purpose** **Members**

Watch

A device that measures time.

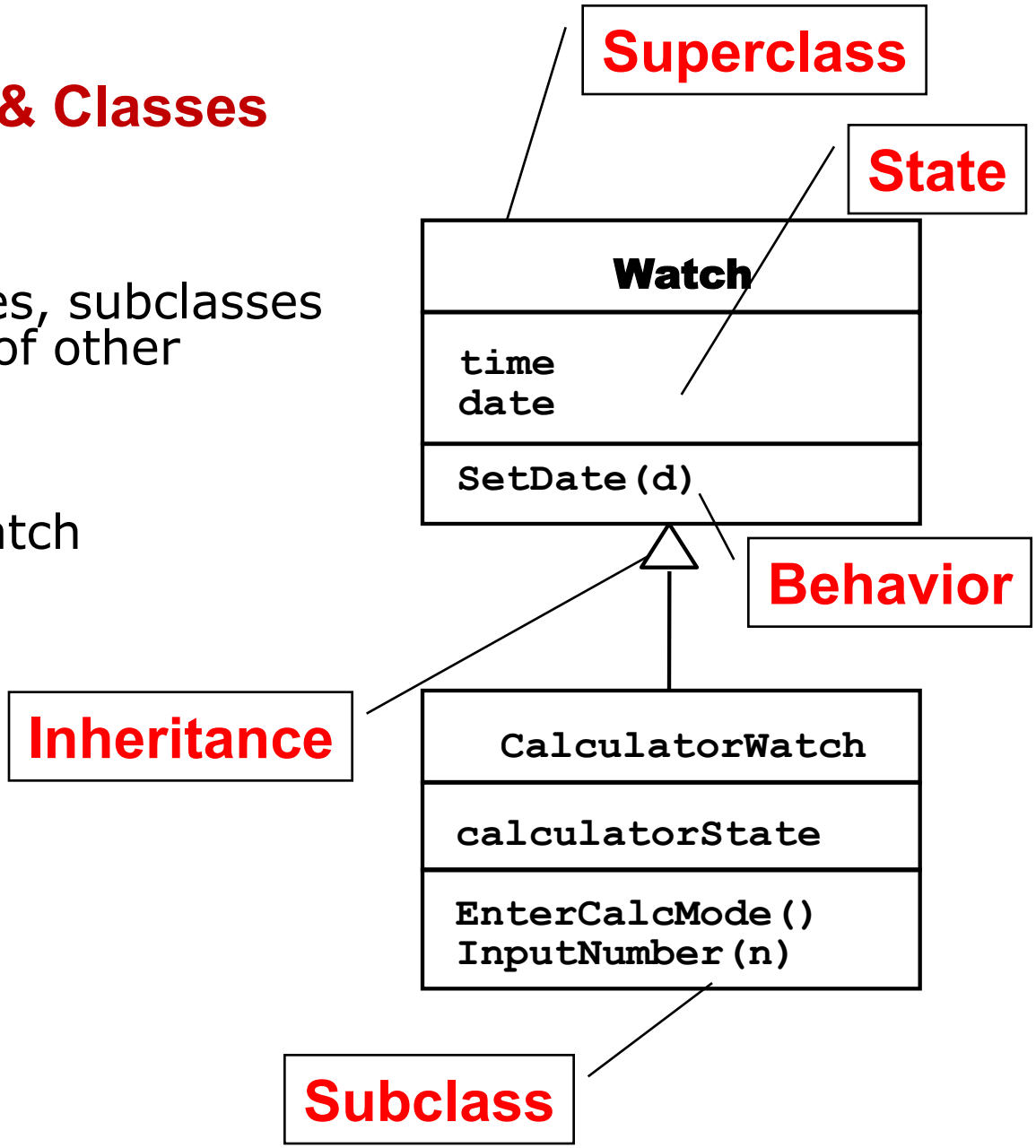


- Definition **Abstraction:**
  - Classification of phenomena into concepts
- Definition **Modeling:**
  - Development of abstractions to answer specific questions about a set of phenomena while ignoring irrelevant details.

Abstract Data Types & Classes

Unlike abstract data types, subclasses can be defined in terms of other classes using inheritance

- Example: Calculator Watch



Type and Instance

- **Type:**
  - A concept in the context of programming languages
  - **Name**: int
  - **Purpose**: integral number
  - **Members:** `0, -1, 1, 2, -2,…`
- **Instance:**
  - Member of a specific type_ 459-8765)
- **The type of a variable represents all possible instances of the variable.**
- **The following relationships are similar:**

  Type <–> Variable (st_no).

  Concept <–> Phenomenon (integer).

  Class <-> Object (student).

Systems

- A *system* is an organized set of communicating parts
  - Natural system: A system whose ultimate purpose is not known نظام غرضه النهائي غير معروف
  - Engineered system: A system which is designed and built by engineers for a specific purpose
- The parts of the system can be considered as systems again
  - In this case, we call them *subsystems*

Systems

- **Examples of natural systems:**
  - Universe, earth, ocean الكون، الأرض، المحيط
- **Examples of engineered systems:**
  - Airplane, watch, GPS
- **Examples of subsystems:**
  - Jet engine, battery, satellite.

Systems, Models, and Views

- A ***model*** is an abstraction describing a system or a subsystem
- A ***view*** depicts selected aspects of a model
- A ***notation*** is a set of graphical or textual rules for depicting models and views:
  - formal notations, "napkin designs"

System: Airplane

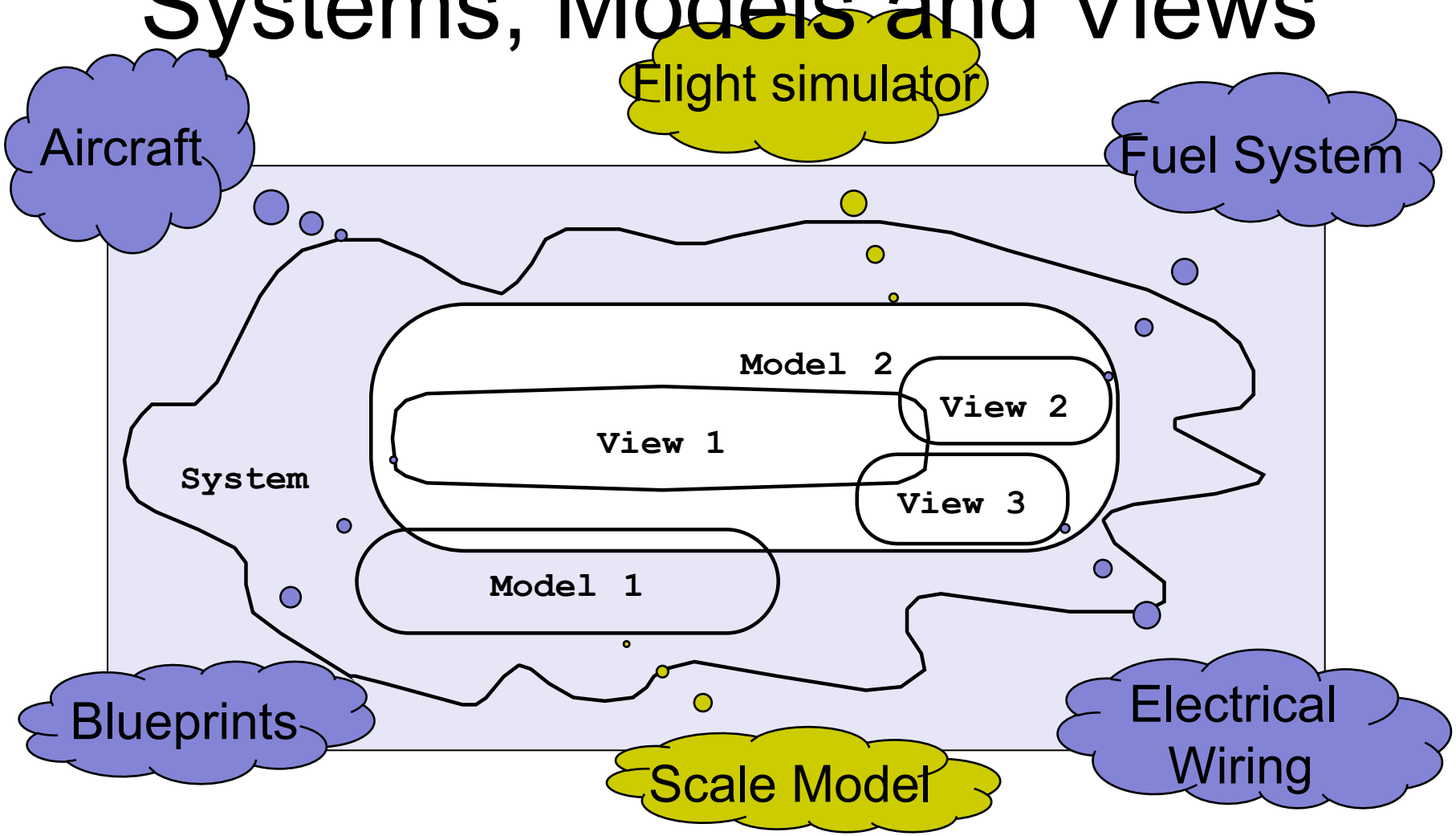
- مخطط مكونات الطائرة
- مخطط الأسلاك الكهربائية، نظام الوقود
- موجة صوتية تم إنشاؤها بواسطة الطائرة

Models:
Flight simulator
Scale model

محاكاة الطيران

Views:
Blueprint of the airplane components
Electrical wiring diagram, Fuel system
Sound wave created by airplane

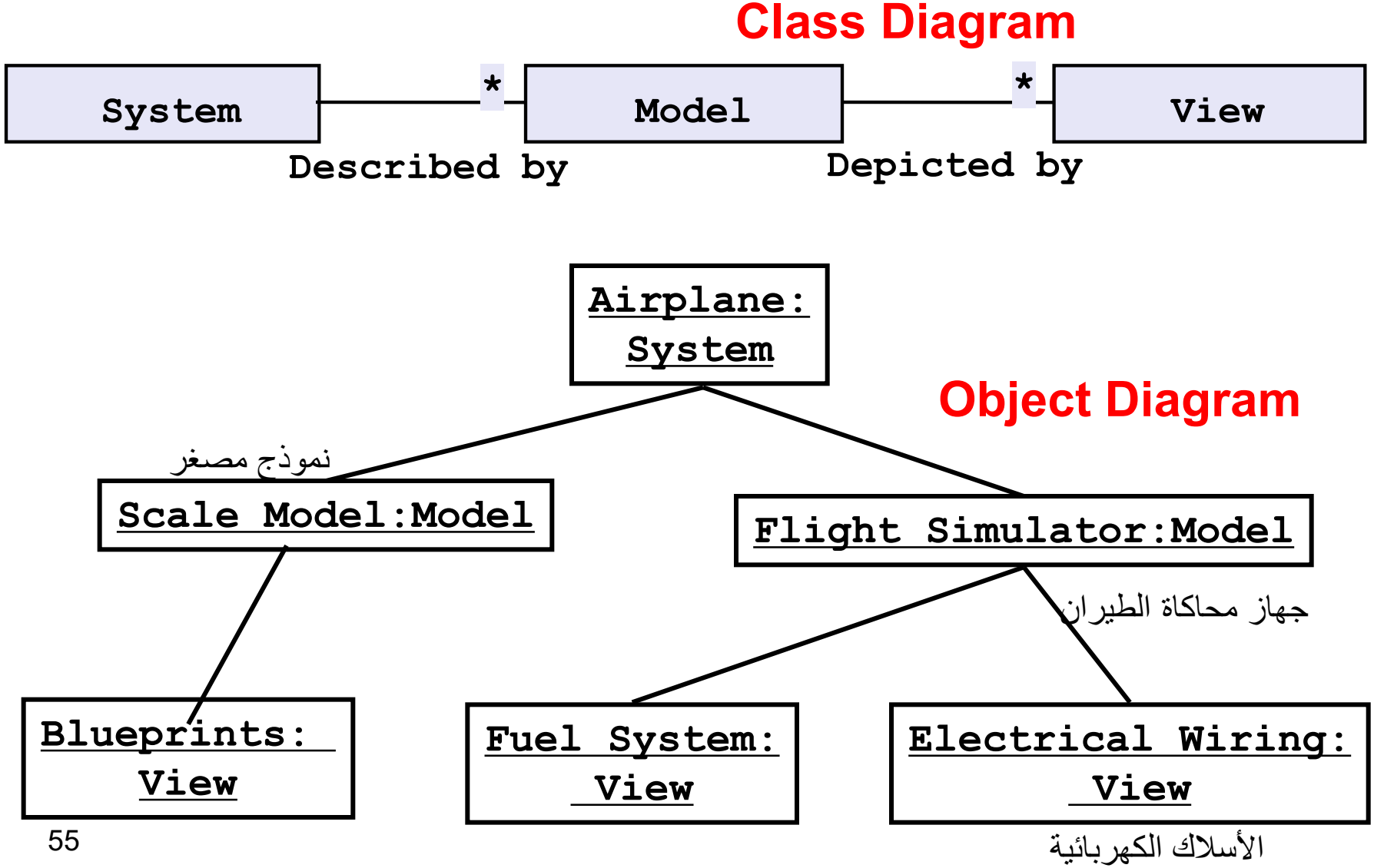
Systems, Models and Views



Views and models of a complex system usually overlap

(UML Notation)

Systems, Models and Views



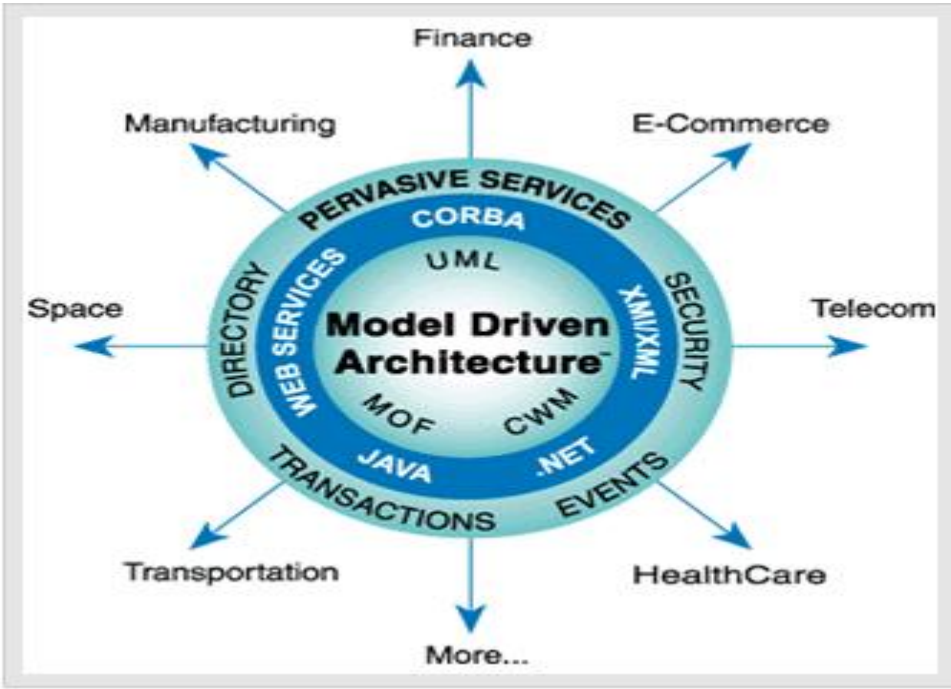
Model-Driven Development

1. Build a platform-independent model of application functionality and behavior
   a) Describe the model in modeling notation (UML)
   b) Convert model into a platform-specific model
2. Generate executable from platform-specific model
3. Advantages:
   – Code is generated from the model (“mostly”)
   – Portability and interoperability.

Model-Driven Architecture effort:
http://www.omg.org/mda/
OMG: Object Management Group

Model Driven Architecture® **(MDA®)** is an approach to software design, development, and implementation spearheaded by the **OMG.**

MDA provides guidelines for structuring software specifications that are expressed as models.



Application vs Solution Domain

- **Application Domain** (Analysis):
  - The environment in which the system is operating.
- **Solution Domain** (Design, Implementation):
  - The technologies used to build the system
- Both domains contain abstractions that we can use for the construction of the system model.

Object-Oriented Software Engineering

Using UML, Patterns, and Java

